

Genomgång - Databasdesign RDBMS

Databasdesign är att bestämma hur datan skall lagras i databasen

Data normalisering

Data normalisering är processen att organisera data i en databas på ett sådant sätt att redundans minimeras och beroenden mellan data elementen minskas. Syftet med data normalisering är att uppnå flera viktiga mål:

1. **Dataintegritet:** Genom att eliminera onödiga redundanser minskas risken för inkonsekvens och felaktigheter i databasen, vilket leder till högre dataintegritet.
2. **Effektivitet:** En väl-normaliserad databas är mer effektiv att fråga, vilket förbättrar prestanda och produktivitet.
3. **Flexibilitet:** Normaliserad data kan lättare anpassas till förändrade krav och skalas när databasen växer.
4. **Klarhet:** En tydligt strukturerad databas, som följer normalisering principerna, är lättare att förstå och underhålla för utvecklare och användare.

Normalform

Normalform är en term inom databasdesign som beskriver graden av normalisering av en databasstruktur. Normalformerna är en serie regler som används för att organisera data i en relationsdatabas på ett effektivt sätt för att minimera redundans och beroenden.

Det finns många normalformer. Dock är det vanligast att implementera normaliseringen upp till 3NF.

Första normalformen (1NF):

- Atomära värden - Kräver att varje cell i en tabell innehåller endast ett enda värde, och samma datatyp genom hela kolumnens alla rader
- Primär nyckel - Rader måste ha en unik kandidatnyckel
- Inga upprepade grupper - Det får inte finnas upprepade grupper, såsom flera kolumner av samma sort (ex. course1,course2,course3)

Andra normalformen (2NF):

- Inga partiella beroenden - Kräver att alla icke-nyckelattribut är fullt funktionsberoende av hela primära nyckeln. Detta innebär att om en tabell har en sammansatt primära nyckeln, ska alla andra attribut vara beroende av hela den sammansatta nyckeln, inte bara en del av dessa.

Tredje normalformen (3NF):

- Eliminera transitiva beroende - Kräver att alla icke-nyckelattribut är beroende av primära nyckeln och inte på andra icke-nyckelattribut. Detta eliminerar transitiva beroenden mellan attribut.

Exempel på en övning: Ett skolsystem har hand om studenter, kurser och lärare. Varje student har (id, personnummer, namn). En student kan ha flera kurser. Varje kurs har (id, kursnamn, antal studenter). En kurs kan ha flera studenter. En kurs kan ha max en lärare. Varje lärare har (id, namn). En lärare kan ha flera kurser.

Normal form exempel

Ej normaliserad tabell

ssn	student_name	course_name	student_count	teacher_name	teacher_phone
950101-XXXX	Erin	JS	4	Gregor	076XXXXX
961128-XXXX	Maria	JS	4	Gregor	076XXXXX
000322-XXXX	August	JS	4	Gregor	076XXXXX
990915-XXXX	Ahmed	JS, PHP, Node	4, 1, 1	Gregor, Pia, Pia	076X, 070X, 070X

Några problem i onormaliserad tabell:

1. Går mot Atomäritet (1NF-fel)

I raden för Ahmed ser vi listor: JS, PHP, Node och Gregor, Pia, Pia.

- Sökproblem: SQL-fråga `WHERE course_name = 'PHP'` kan inte hitta Ahmed, eftersom hans cell innehåller hela strängen "JS, PHP, Node".
- Sorteringsproblem: Det går inte att sortera tabellen efter kursnamn på ett vettigt sätt när en cell innehåller tre olika kurser.

2. Matchningskaos

Data hänger bara ihop genom sin position i listan.

- Problemet: Vi måste "anta" att den andra kursen (PHP) hör ihop med den andra läraren (Pia) och det andra telefonnumret (070X).
- Risker: Om någon råkar skriva `PHP, JS2, Node` men glömmer att byta ordning på lärarna, kopplas Ahmed plötsligt till fel lärare för fel kurs.

3. Beräkningsproblem

- Aggregering: Om du vill räkna ut det totala antalet kursdeltagare med `SUM(student_count)`, kommer databasen att krascha eller ge felmeddelande eftersom "4, 1, 1" inte är ett tal, utan en textsträng. Du kan inte göra matematik på kommatecken.

4. Svårhanterliga uppdateringar

Tänk dig att Ahmed hoppar av bara PHP-kursen.

- Problemet: Du kan inte bara ta bort en rad. Du måste skriva kod som hämtar strängen "JS, PHP, Node", klipper ut "PHP", städar bort rätt kommatecken och sedan sparar tillbaka "JS, Node".

1NF Exempel

- Alla celler får endast ha ett värde
- Raden måste ha en unik kandidatnyckel. OBS! Behöver ej ha en primärnyckel, kan lösas med att ha en sammansatt nyckel såsom det visas med de orangea kolumnerna
- Du får inte skapa kolumner som `course1`, `course2`, `course3` för att gå runt problemet med listor. Om en student läser tre kurser ska de ligga på tre rader, inte i tre numrerade kolumner.

ssn	student_name	course_name	student_count	teacher_name	teacher_phone
950101-XXXX	Erin	JS	4	Gregor	076XXXXX
961128-XXXX	Maria	JS	4	Gregor	076XXXXX
000322-XXXX	August	JS	4	Gregor	076XXXXX
990915-XXXX	Ahmed	JS	4	Gregor	076XXXXX
990915-XXXX	Ahmed	PHP	1	Pia	070XXXXX
990915-XXXX	Ahmed	Node	1	Pia	070XXXXX

Några problem med 1NF exemplet:

1. Redundans (Onödig upprepning)

Varje gång en student läser en kurs måste du skriva in all information igen.

- Exempel: Information om "JS"-kursen, att den har 4 deltagare, att Gregor är lärare och hans telefonnummer, skrivs ut fyra gånger.

2. Uppdatering

Tänk dig att läraren Gregor byter telefonnummer.

- Problemet: Du kan inte bara ändra på ett ställe. Du måste leta upp alla rader där Gregor förekommer (i det här fallet 4 rader) och uppdatera numret på varje rad.

3. Insättning

Tänk dig att skolan skapar en ny kurs, "Python", men inga studenter har hunnit anmäla sig.

- Problemet: Eftersom primärnyckeln i denna tabell kräver ett ssn (personnummer), kan du inte lägga in kursen i systemet. Du kan inte spara information om kursen, läraren eller telefonnumret förrän du har minst en student.

4. Radering

Tänk dig att Ahmed bestämmer sig för att hoppa av kurserna "PHP" och "Node".

- Problemet: Om du raderar dessa två rader för att Ahmed slutat, försvinner all information om att kurserna PHP och Node någonsin har funnits.

2NF Exempel

- Måste uppfylla alla 1NF krav
- Kräver att alla icke-nyckelattribut är fullt funktionsberoende av hela kandidatnyckel. Detta innebär att om en tabell har en sammansatt kandidatnyckel, ska alla andra attribut vara beroende av hela den sammansatta nyckeln, inte bara en del av den.

students

id (PK)	ssn	name
1	950101-XXXX	Erin
2	961128-XXXX	Maria
3	000322-XXXX	August
4	990915-XXXX	Ahmed

courses

id (PK)	name	student_count	teacher_name	teacher_phone
1	JS	4	Gregor	076XXXXX
2	PHP	1	Pia	070XXXXX
3	Node	1	Pia	070XXXXX

students_courses (Visar på relationen mellan Student & Kurs tabellerna)

student_id (FK)	course_id (FK)
1	1
2	1
3	1
4	1
4	2
4	3

Many-to-Many

students → students_courses → courses

Exempel på problem i 2NF exemplet:

1. Uppdatering

I din 2NF-tabell ligger informationen om läraren direkt i courses.

- Problemet: Om Pia byter telefonnummer och hon ansvarar för 10 olika kurser, måste du uppdatera 10 olika rader.

2. Insättning

- Problemet: Du kan inte lägga in en ny lärare i systemet förrän hen har blivit tilldelad en kurs.

3. Radering

Detta är ofta det farligaste problemet.

- Problemet: Om du raderar en kurs för att den ställs in, råkar du också radera informationen om läraren.
- Exempel: Om "Node"-kursen tas bort för att ingen anmält sig, och Pia var den enda som undervisade i den, så försvinner alla spår av Pia ur systemet. Du förlorar data du egentligen vill ha kvar (lärarens kontaktuppgifter).

3NF Exempel

- Måste uppfylla alla 2NF krav
- Eliminera transitive beroenden:
“Every non-key attribute must provide a fact about the key, the whole key, and nothing but the key (so help me Codd).”

students

id (PK)	ssn	name
1	950101-XXXX	Erin
2	961128-XXXX	Maria
3	000322-XXXX	August
4	990915-XXXX	Ahmed

students_courses

student_id (FK)	course_id (FK)
1	1
2	1
3	1
4	1
4	2
4	3

courses

id (PK)	course	student_count	teacher_id (FK)
1	JS	4	1
2	PHP	1	2
3	Node	1	2

teachers

id (PK)	name	phone
1	Gregor	076XXXXX
2	Pia	070XXXXX

Läsanvisningar:

- [DB Normalisering](#)
- [ER-diagram](#)
- [Kan använda draw.io för att rita ER-diagram](#)